Chapter 7: Integer Arithmetic

Shift and Rotate Instruction:

- SHL = shift left
- SHR = shift right
- SAL = shift arithmetic left
- SAR = shift arithmetic right
- ROL = rotate left
- ROR = rotate right
- RCL = rotate carry left
- RCR = rotate carry right
- SHLD = double precision shift left
- SHRD = double precision shift right

- Logical Shift:

Left Shift:
the bits from lsb to msb are moved towards their left the msb is stored in carry flag
the empty space of lsb is filled with 0

Right Shift:
the bits from msb to lsb are moved towards their right the lsb is stored in carry flag
the empty space of msb is filled with 0

- Arithmetic Shift:

Left Shift:
the bits from lsb to msb are moved towards their left the msb is stored in carry flag
the empty space of lsb is filled with the copy of lsb i.e the value of lsb

Right Shift:
the bits from msb to lsb are moved towards their right the lsb is stored in carry flag
the empty space of msb is filled with the cpy of msb i.e the value of msb

- Syntax for SHL, SHR, SAL, SAR

command destination, count

- shl can be used to perform bitwise multiplication and shr can be used to perform
bitwise division

- sar and sal can be used to perform signed division and multiplication

- SHLD Instruction:
- takes the destination, source and count as input and shift count number of bits to the left
from source to destination

- the source operand is not affected but the sign, zero, auxiliary flag, parity and carry
  flags are affected.

- destination's msb is copied in carry flag while source's msb becomes the lsb of destination

- SHRD Instruction:
  - takes the destination, source and count as input and shift count number of bits to the right
    from souce to destination

  - the source operand is not affected but the sign, zero, auxiliary flag, parity and carry
    flags are affected.

  - the destination's lsb is copied in carry flag while the source lsb becomes destination's msb

- Allowed Syntax for SHLD/SHRD:

```
command register16, register16, cl/imm8
command mem16, register16, cl/imm8
command register32, register32, cl/imm8
command mem32, register32, cl/imm8
```

Rotation:

ROL Instruction:
    all the bits are moved towards their left the value of msb is copied in carry flag
    aswell as the vacant space of lsb

ROR Instruction:
    all the bits are moved towards their right the value of lsb is copied in carry flag
    aswell as the vacant space of msb

RCL Instruction:
    all the bits are moved towards their left the value of carry flag is copied in lsb
    and the value of msb is moved to carry flag

RCR Instruction:
    all the bits are moved towards their right the value of carry flag is copied in msb
    and the value of lsb is moved to carry flag

Binary Multiplication:

- MUL Instruction:
    3 versions of MUL:
        1. multiplies an 8-bit operand by the al register
        2. multiplies a 16-bit operand by the ax register
        3. multiplies a 32-bit operand by the eax register

- Allowed Syntax:
  see photo 1.

- mul sets the carry and overflow flag if the upper half of the product is not equals to 0

- IMUL Instruction:

  - performs signed binary multiplication
  - preserves the sign of the product by sign extending the highest bit of the lower half
    of the product into the upper bits of the product

  - Single operand format:

    the one operand formats store the product in dx:ax or edx:eax
    the carry and overflow flags are set if the upper half of the product is not a
    sign extension of the lower half

  - Two operand format:

    it stores the operand in the first operand, which must be a register. The second
    operand can be a register, a memory operand, or an immediate value
    the two operand formats truncate the product to the length of the destination

    if significant digits are lost the overflow and carry flags are set

  - Three operand format:
    The three-operand formats in 32-bit mode store the product in the first
    operand. The second operand can be a 16-bit register or memory operand,
    which is multiplied by the third operand, an 8 or 16-bit immediate value

    If significant digits are lost when IMUL executes, the Overflow and Carry flags
    are set.

Binary Division:

- DIV Instruction:
  see photo 2.

  - Signed Integer Division:
    signed integer division is nearly identical to unsigned division, with one
    important difference: the dividend must be sign extended before the division
    takes place

Signed Extension Instruction: CBW, CWD, CDQ:
the cbw instruction extends the sign bit of al into ah, preserving the number's sign

The CWD (convert word to doubleword) instruction extends the sign bit of AX
into DX

The CDQ (convert doubleword to quadword) instruction extends the sign bit of
EAX into EDX

Extended Addition:
extended precision addition is the technique of adding and subtracting numbers having
an almost unlimited size

ADC Instruction:
the adc (add with carry) instruction add both a source operand and the contents of the
carry flag to a destination operand.

same syntax as add instruction

SBB Instruction:
the sbb instruction subtracts both a source operand and the value of the carry flag from
the destination operand